中山大学计算机学院本科实验报告

（2025 学年春季学期）

课程名称:	并行程序设计与算法
实验题目:	Lab4 基于 Pthreads 的并行方程求解与蒙特卡洛算法
学号:	23336128
姓名:	梁力航
完成日期:	2026 年 4 月 23 日

本实验使用 Pthreads 完成两个并行程序：其一通过条件变量协调判别式计算线程与两个求根线程，求解一元二次方程；其二将蒙特卡洛采样任务分配给多个线程，使用线程局部随机数种子估计圆周率。实验重点不只是给出正确结果，还包括分析线程创建、条件变量同步、随机数生成与任务粒度对性能的影响。

摘要

本实验围绕 Pthreads 编程模型实现两个典型任务：并行求解一元二次方程与蒙特卡洛方法估计圆周率。方程求解任务采用条件变量构造显式任务依赖，使两个求根线程在判别式 $\Delta = b^2 - 4ac$ 计算完成后继续执行。蒙特卡洛任务将总采样数 n 静态划分到多个线程，每个线程使用 `rand_r()` 维护独立随机数状态，最后由主线程汇总落入四分之一圆的点数 m，得到 $\pi \approx 4m/n$。实验结果表明，细粒度方程求解任务主要受线程创建和同步开销支配；在题目给定采样规模内，蒙特卡洛任务可以展示一定并行收益，但线程创建与调度开销仍会明显影响加速比。

1 实验目的

本实验目标如下：

1. 使用 Pthreads 条件变量实现线程间同步，完成一元二次方程求根；
2. 使用 Pthreads 对蒙特卡洛采样任务进行线程级并行划分；
3. 使用线程安全的随机数生成方式，避免多个线程共享 `rand()` 全局状态；
4. 记录不同线程数下的运行时间、估计值与加速比，并分析线程创建和同步开销。

2 Algorithm Design

2.1 并行一元二次方程求解

一元二次方程 $ax^2 + bx + c = 0$ 的根由公式

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

给出。由于两个根都依赖判别式 $\Delta = b^2 - 4ac$，本实验将任务拆为三个线程：

- 线程 A 计算判别式 Δ，写入共享状态后调用 `pthread_cond_broadcast()`；
- 线程 B 等待条件变量，收到通知后计算 x_1；
- 线程 C 等待条件变量，收到通知后计算 x_2。

算法 1 基于条件变量的一元二次方程并行求解

初始化互斥锁、条件变量与共享标志 *delta_ready* $\leftarrow false$
创建两个求根线程 T_1, T_2，二者进入条件变量等待
创建判别式线程 T_Δ
T_Δ 计算 $\Delta = b^2 - 4ac$
T_Δ 加锁，写入 Δ，设置 *delta_ready* $\leftarrow true$
T_Δ 广播条件变量，唤醒 T_1, T_2
T_1, T_2 分别计算 $\frac{-b+\sqrt{\Delta}}{2a}$ 与 $\frac{-b-\sqrt{\Delta}}{2a}$
主线程 join 三个线程并输出根与运行时间

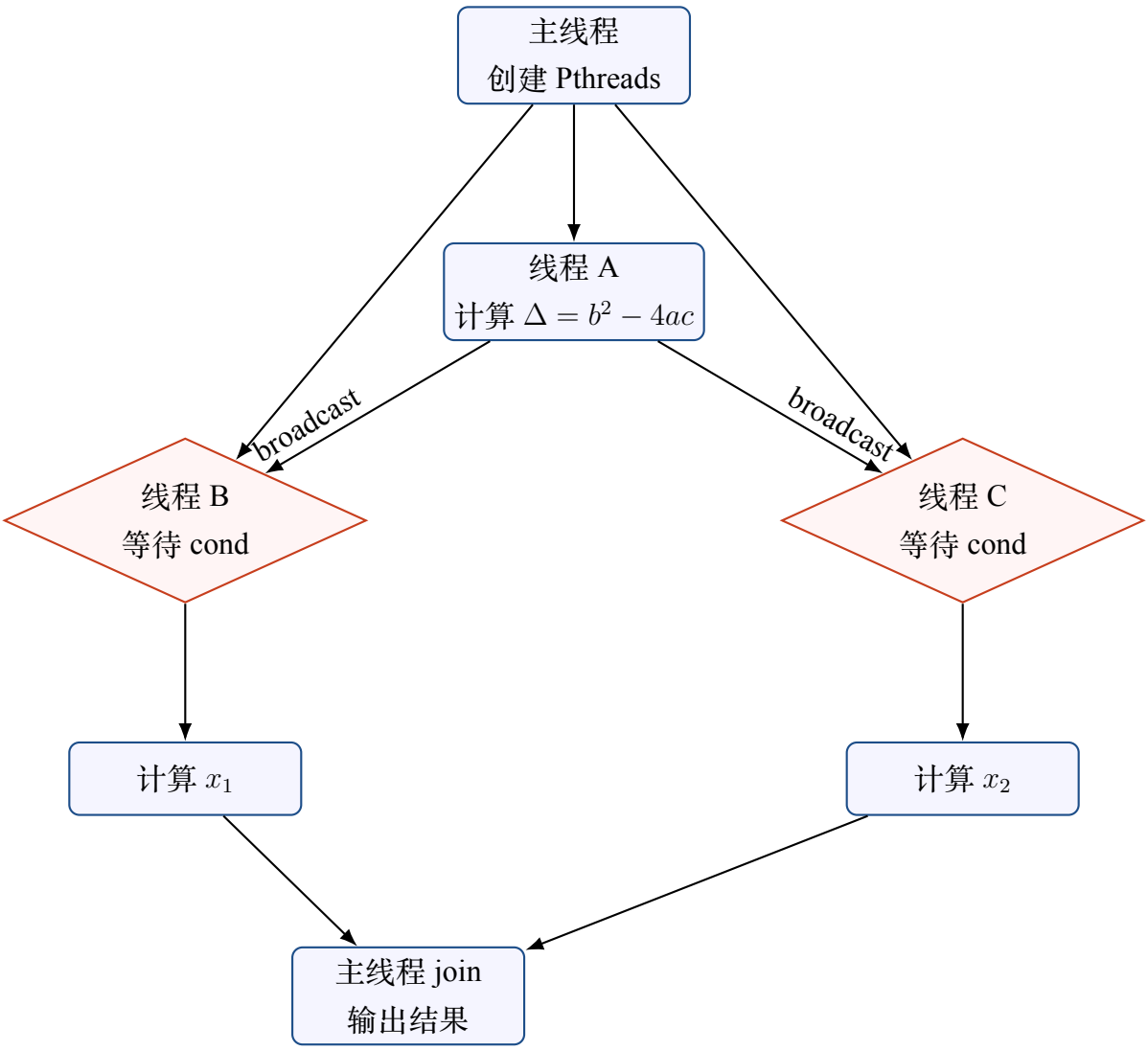

图 1: 一元二次方程求解任务依赖图

2.2 蒙特卡洛方法估计圆周率

蒙特卡洛方法在单位正方形 $[0,1]\times[0,1]$ 内生成 n 个随机点。若点满足 $x^2+y^2\le 1$，则它位于单位圆的四分之一圆内。设命中点数为 m，则

$$\pi \approx 4\frac{m}{n}.$$

并行版本将 n 按线程数 p 静态切分，每个线程只维护自己的命中计数，最后由主线程汇总。为了避免 `rand()` 的全局状态竞争，每个线程使用独立 seed 调用 `rand_r()`。

算法 2 Pthreads 蒙特卡洛估计圆周率

输入总采样数 n、线程数 p、随机种子 s
将 n 划分为 p 份，余数分配给前若干线程
for 每个线程 i **do**
    使用独立 seed 初始化线程局部随机数状态
    生成 $(x,y)\in[0,1]^2$，统计 $x^2+y^2\le 1$ 的次数 m_i
end for
主线程 join 所有线程，计算 $m=\sum_i m_i$
输出 $n, m, \pi = 4m/n$ 与运行时间

3 Implementation Details

3.1 条件变量同步实现

代码 1 展示了方程求解程序的关键同步逻辑。求根线程使用 `while` 循环检查共享标志，而不是只用一次 `if`，这是条件变量的常规写法，可抵御虚假唤醒。判别式线程完成计算后先在互斥锁保护下写入共享状态，再广播条件变量唤醒两个求根线程。

代码 1: 条件变量同步核心代码

```
pthread_mutex_lock(&state->mutex);
while (!state->delta_ready) {
    pthread_cond_wait(&state->cond, &state->mutex);
}
const double delta = state->delta;
pthread_mutex_unlock(&state->mutex);

pthread_mutex_lock(&state->mutex);
state->delta = delta;
state->delta_ready = true;
pthread_cond_broadcast(&state->cond);
```

```
pthread_mutex_unlock(&state->mutex);
```

完整源码位于 lab/lab4/src/quadratic_solver_pthreads.c。程序支持浮点数输入，例如：

```
./bin/quadratic_solver_pthreads 1 -3 2
```

输出保持稳定的 key=value 形式，包括 delta、x1、x2 和 time_sec。

3.2 线程局部随机数实现

代码 2 展示了蒙特卡洛任务的核心循环。每个线程拥有独立 seed，调用 rand_r() 时只更新自己的局部状态，因此不需要在随机数生成阶段加锁。每个线程也只写入自己的 inside 字段，最终汇总在所有线程 join 之后完成。

代码 2: 蒙特卡洛线程局部采样核心代码

```c
static void *monte_carlo_worker(void *arg) {
    MonteCarloTask *task = (MonteCarloTask *)arg;
    unsigned int seed = task->seed;
    uint64_t inside = 0;

    for (uint64_t i = 0; i < task->samples; ++i) {
        const double x = unit_random(&seed);
        const double y = unit_random(&seed);
        if (x * x + y * y <= 1.0) {
            ++inside;
        }
    }

    task->inside = inside;
    task->seed = seed;
    return NULL;
}
```

完整源码位于 lab/lab4/src/monte_carlo_pi_pthreads.c。程序使用方式如下：

```
./bin/monte_carlo_pi_pthreads 65536 8 20250401
```

4 Performance Analysis

4.1 实验环境与运行方式

程序使用如下命令编译。计时使用单调时钟，统计范围覆盖线程创建、线程计算、线程 join 与结果汇总，因此表格反映的是端到端运行时间。

```
make -C lab/lab4
./lab/lab4/bin/quadratic_solver_pthreads 1 -3 2
./lab/lab4/bin/monte_carlo_pi_pthreads 65536 8 20250401
```

4.2 实验结果

表 1 给出了题目输入范围上限 $n = 65,536$ 时蒙特卡洛程序在不同线程数下的单次运行结果。加速比以 1 线程时间为基准计算。

表 1: 蒙特卡洛估计 π 的运行时间与加速比

线程数	采样数 n	命中数 m	时间 t / s	加速比
1	65536	51445	0.000719	1.000
2	65536	51414	0.000384	1.872
4	65536	51419	0.000258	2.787
8	65536	51524	0.000253	2.842

表 2 给出方程求解程序的代表性输出。由于该任务固定由 3 个线程完成，并且单次计算量极小，其运行时间主要由线程创建、条件变量等待和调度开销构成。

表 2: 一元二次方程并行求解代表性结果

a	b	c	Δ	根类型	输出结果
1	-3	2	1	实根	$x_1 = 2,\ x_2 = 1,\ t \approx 1.50 \times 10^{-4} s$
1	2	5	-16	复根	$x_1 = -1 + 2i,\ x_2 = -1 - 2i,\ t \approx 2.69 \times 10^{-4} s$

4.3 性能讨论

对一元二次方程求解而言，并行化的教育意义大于性能意义。计算 Δ 和两个根都只是常数级浮点运算，而创建 3 个线程、维护互斥锁、进入条件变量等待、广播唤醒和上下文调

度都需要额外成本。因此该任务通常不会比串行公式求解更快。它适合作为条件变量同步练习，而不适合作为追求加速比的计算任务。

对蒙特卡洛估计 π 而言，每个采样点之间相互独立，任务具有天然数据并行性。表 1 中 2、4、8 线程相对单线程均有加速，但由于题目给定的最大采样数只有 $65,536$，单次计算时间已经进入亚毫秒量级，线程创建、join 和操作系统调度开销占比较高。因此 4 线程到 8 线程的收益明显收敛，实际加速比低于理想线性值。若进一步扩大采样规模，固定线程开销会被更多随机采样计算摊薄；但在本实验题面范围内，更合理的结论是：该任务具备并行结构，实际收益受任务粒度限制。

本实验没有在采样循环中使用全局锁。每个线程只写自己的局部计数，主线程在 join 后汇总，因此同步开销集中在线程生命周期边界。这种设计符合 KISS 和 DRY 原则：并行路径简单、共享状态最小、无需引入复杂的锁竞争控制。

5 Conclusion

本实验完成了两个 Pthreads 并行程序和对应报告源码。方程求解任务展示了条件变量的正确使用方式：共享状态必须由互斥锁保护，等待端必须循环检查谓词，生产端写入状态后再广播唤醒。蒙特卡洛任务展示了更适合并行化的数据并行结构：采样任务彼此独立，只需在线程结束后进行一次归约。

实验结论是：并行程序设计不能只看“是否使用多线程”，还要判断任务粒度是否足够大、同步是否必要、共享状态是否可避免。对于极小计算任务，多线程开销可能显著超过计算本身；对于可分解的大规模独立采样任务，Pthreads 可以有效提升吞吐量。

附录：源码文件

本实验源码组织如下：

- `lab/lab4/src/quadratic_solver_pthreads.c`：条件变量同步的一元二次方程求解；
- `lab/lab4/src/monte_carlo_pi_pthreads.c`：线程局部随机数的蒙特卡洛估计 π；
- `lab/lab4/Makefile`：统一构建入口。

提交 PDF 时，可按课程要求命名为 `并行程序设计 _ 学号 _ 姓名.pdf`。